

METODOLOGÍA I

Trabajo Práctico 2 — LLM, herramientas de IA y tokenización en la práctica

OBJETIVO GENERAL

Aplicar de forma técnica y verificable los conceptos centrales de la Unidad 1 sobre cómo funciona un modelo de lenguaje: qué es (y qué NO es) un LLM, la diferencia entre marca/proveedor y modelo, la diferencia entre categorías de herramientas de asistencia al desarrollo (asistente conversacional vs. agente de terminal), la ventana de contexto —incluyendo attention decay y compactación—, las alucinaciones y la tokenización.

A diferencia del Trabajo Práctico 1 (de reflexión sobre el impacto de la IA en la profesión), este TP es un **laboratorio técnico**: cada consigna se resuelve probando el concepto contra un modelo o una herramienta real, no solo describiéndolo de memoria. Retoma en particular el "Laboratorio práctico: tokens y modelos en la mano" de la Actividad 3 y lo lleva a un entregable completo y evaluable.

Material de referencia obligatorio: antes de resolver, repasá el contenido de las tres Actividades de esta unidad — la Actividad 1 (qué es un LLM), la Actividad 2 (capacidades, alucinaciones, panorama de herramientas) y la Actividad 3 (tokens, inferencia, ventana de contexto, laboratorio de nombrar modelos).

MARCO TEÓRICO

Concepto	Aplicación en este TP
LLM (Large Language Model)	Modelo generativo entrenado sobre lenguaje que produce contenido nuevo token por token, a diferencia de un sistema clasificador o de reglas fijas.
Marca/proveedor vs. modelo	La empresa (Anthropic, OpenAI, Google) no es lo mismo que el producto puntual (Claude Opus, GPT-4o, Gemini Pro) — cada modelo tiene su propio tier de capacidad y costo.
Categorías de herramientas de IA	Asistente conversacional de propósito general, autocompletado de editor, agente de terminal, herramienta de Specification Driven Development — cada una con un grado distinto de autonomía. Criterio simple: sin tools es chat, con tools es agente.
Ventana de contexto	Cantidad de información, en tokens, que el modelo puede "tener a la vista" en un momento dado.
Attention decay	Degradación de calidad a medida que la ventana de contexto se llena, aunque el contenido todavía entre — el modelo no falla de forma visible, pierde precisión de forma silenciosa (ej. "lost in the middle").
Compactación	Resumen forzado de la conversación cuando se acerca al límite de contexto, para poder seguir — se pierden detalles finos que no entran en el resumen.

Concepto	Aplicación en este TP
Alucinación	Afirmación incorrecta producida con la misma confianza discursiva que una correcta — no se distingue por el tono, solo verificando contra una fuente real.
Tokenización	Proceso por el cual un modelo divide el texto en tokens (no en palabras) — la unidad real de costo y de límite de contexto.

CASO PRÁCTICO

Resolvé las seis partes en orden. Cada una pide un procedimiento concreto y evidencia (captura de pantalla, link o cita textual) — no alcanza con una respuesta de opinión suelta como en el TP1.

Parte 1 — ¿Qué es (y qué NO es) un LLM?

Redactá, con tus propias palabras y en no más de 8 líneas, qué distingue a un LLM de otros sistemas de software que también "responden" pero no son modelos generativos.

Después, clasificá estos tres sistemas como **"LLM generativo"** o **"no es un LLM"**, justificando cada uno en una oración con el criterio visto en la unidad (produce contenido nuevo, token por token, vs. responde según una categoría o regla fija):

- a. Un corrector ortográfico que subraya palabras mal escritas comparándolas contra un diccionario fijo.
- b. Un chatbot de atención al cliente de un banco que solo responde siguiendo un árbol de opciones predefinido ("Escribí 1 para saldo, 2 para reclamos...").
- c. Un asistente tipo ChatGPT o Claude al que le pedís que te explique un fragmento de código.

Ahora el lado inverso: cuatro afirmaciones circulan como sentido común sobre estas herramientas y las cuatro son **falsas**. Para cada una, escribí en 2-3 líneas *por qué* es falsa, apoyándote en un concepto técnico ya visto en la unidad (no alcanza con "porque no", tiene que quedar claro el mecanismo real detrás):

- d. **"El modelo guarda hechos en una base de datos interna y los consulta."** (pista: relacionalo con cómo se explica una alucinación — el modelo genera texto a partir de patrones estadísticos, no consultando una fuente de verdad.)
- e. **"Si le contás algo importante en el chat, el modelo lo va a recordar la próxima vez que lo abras."** (pista: memoria temporal vs. memoria permanente, Actividad 3.)
- f. **"El modelo entiende lo que le pedís, igual que lo entendería una persona."** (pista: "usar un LLM" vs. lo que realmente hace un modelo generativo, Actividad 1 — completa patrones aprendidos, no comprende en el sentido en que comprende un humano.)
- g. **"Si le hacés la misma pregunta dos veces, te tiene que dar exactamente la misma respuesta — si no, algo está mal."** (pista: temperatura e inferencia probabilística, Actividad 3.)

Parte 2 — Marca vs. modelo: por qué no hay "el mejor" modelo

En la Actividad 3 hiciste el ejercicio de inferir, solo por la convención de nombres, qué tier de capacidad y costo tiene cada modelo (Claude Opus/Sonnet/Haiku, GPT, Gemini Pro/Flash). Este punto profundiza esa misma idea con un eje que ahí quedó implícito: la diferencia entre **proveedor/marca** (la empresa: Anthropic, OpenAI, Google) y **modelo** (el producto puntual de esa empresa: Claude Opus, GPT-4o, Gemini Pro).

a. Completá esta tabla con al menos **tres proveedores** distintos, cada uno con **dos o más modelos** reales de su catálogo actual (buscalo en la documentación oficial de cada uno, citando la fuente):

Proveedor / marca	Modelo(s)	Tier inferido (premium / medio / liviano)
[Completar]	[Completar]	[Completar]
[Completar]	[Completar]	[Completar]
[Completar]	[Completar]	[Completar]

- b. Con esa tabla armada, respondé: para estas tres tareas, ¿qué tier de modelo elegirías para cada una y por qué? No hace falta nombrar un producto específico, alcanza con justificar en términos de capacidad vs. costo:
- Diseñar la arquitectura de un sistema nuevo, con varias alternativas a evaluar y trade-offs no obvios.
 - Generar cientos de mensajes de commit cortos y repetitivos a partir de un diff.
 - Autocompletar código mientras escribís, en tiempo real, dentro del editor.
- c. Cerrá con una idea propia, en 2-3 líneas: ¿por qué "¿cuál es el mejor modelo?" es, en general, una pregunta mal planteada? Reformulá esa pregunta en una versión mejor.

Fuente de apoyo para este punto: material de referencia transversal del curso sobre modelos y selección de motor por tarea ("cada tarea tiene su motor ideal" — no existe un único "mejor" modelo, sino un tier más adecuado según capacidad, costo y el tipo de tarea).

Parte 3 — Chatbot vs. agente: la categoría y el criterio de las "tools"

La Unidad 1 presentó un panorama de categorías de herramientas de asistencia al desarrollo: asistentes conversacionales de propósito general, autocompletado integrado al editor, agentes de terminal capaces de ejecutar comandos y modificar archivos, y herramientas de Specification Driven Development. *(La distinción completa "chatbot vs. agente" —el loop observar-planificar-actuar-verificar— se profundiza recién en la Unidad 4; acá el alcance es identificar la categoría de la herramienta con un criterio simple, no ese loop completo.)*

Ese criterio simple es: una herramienta que solo devuelve texto es un **chat**; una herramienta que además puede **actuar sobre el entorno** —ejecutar comandos de terminal, editar archivos, buscar información, guardar memoria — usando capacidades externas ("tools") es un **agente**. Sin tools, es chat. Con tools, es agente.

Clasificá cada una de estas cinco herramientas en su categoría, y para cada una nombrá **qué "tool" concreta usa (si usa alguna)** para actuar sobre el entorno, más allá de generar texto:

1. ChatGPT o Claude, usados desde el chat web.

2. GitHub Copilot en su modo de autocompletado dentro del editor de código.
3. Un agente de terminal (por ejemplo Claude Code, Cursor en modo agente, o similar) al que se le pide "dejame el proyecto sin errores de lint".
4. GitHub Copilot Chat (la ventana de chat dentro del editor, no el autocompletado).
5. Una herramienta de Specification Driven Development como OpenSpec.

Parte 4 — Ventana de contexto: números, attention decay y compactación

Buscá en la documentación oficial de **un** modelo real (por ejemplo, Claude o GPT) cuál es su ventana de contexto en tokens, y citá la fuente (nombre del modelo + URL o documento oficial consultado).

Con ese número, resolvé:

- a. Estimá cuántas páginas de texto (tomá como referencia ~500 palabras por página) entrarían aproximadamente en esa ventana de contexto, mostrando el cálculo que usaste.
- b. Explicá qué pasa, en una conversación larga con ese modelo, cuando el contenido acumulado supera ese límite — conectá tu respuesta con la distinción entre memoria temporal y memoria permanente vista en la Actividad 3.

Ahora un matiz que no depende de desbordar el límite: aunque todo el contenido entre dentro de la ventana de contexto, el modelo no le presta la misma atención a todo por igual. El video 2 de la Actividad 3 ya cita el estudio real "*Lost in the Middle*" (Liu et al., 2023): cuando la información relevante está en el medio de un contexto largo, el modelo la recupera peor que cuando está al principio o al final. Fuera del ámbito académico, a este tipo de degradación de calidad a medida que la ventana se llena —sin que el modelo "falle" de forma visible, el output simplemente empieza a perder precisión aunque siga sonando fluido y seguro— se lo suele nombrar de forma informal como "**attention decay**".

- c. Con tus propias palabras, explicá la diferencia entre "el contenido no entra en la ventana de contexto" (lo que resolviste en el punto b) y "el contenido entra, pero el modelo le presta menos atención según dónde está" (attention decay / lost in the middle). Son dos problemas distintos, aunque los dos tienen que ver con el tamaño del contexto.
- d. Consecuencia práctica: si tenés que escribirle a un modelo un archivo de reglas largo con varias instrucciones, ¿en qué parte del archivo pondrías la instrucción más crítica, según lo que acabás de explicar? Justificá.

Por último, un mecanismo relacionado que **no** forma parte del núcleo del programa de esta unidad, pero que es consecuencia directa de la ventana de contexto llena y vale la pena conocer: cuando una conversación larga se acerca al límite, muchas herramientas reales (chats y agentes por igual) no cortan de golpe — generan un **resumen comprimido** de lo conversado hasta ahí y descartan los mensajes originales, para poder seguir. A ese mecanismo se lo llama **compactación**, y tiene un costo real: el resumen es más chico que la conversación completa, así que se pierden detalles finos —nombres exactos de variables, decisiones de diseño puntuales, preferencias que dijiste de pasada— que no siempre entran en el resumen.

- e. Buscá en la documentación oficial de **una** herramienta real de asistente de IA o agente (por ejemplo, cómo maneja el llenado de contexto Claude, ChatGPT, o un agente de terminal como Claude Code) qué hace cuando

la conversación se acerca al límite de su ventana de contexto. Citá la fuente y resumí en 3-4 líneas qué encontraste.

- f. Conectá esto con la Actividad 3: ¿por qué la compactación es, en el fondo, el mismo problema de memoria temporal vs. permanente visto ahí, aplicado en el momento justo antes de que la información se pierda del todo?

Fuente de apoyo para "attention decay" y "compactación": Liu, N. F. et al., "Lost in the Middle: How Language Models Use Long Contexts" (2023), <https://arxiv.org/abs/2307.03172> (ya citado en el guion del Video 2 de esta Actividad); y material de referencia transversal del curso sobre ventana de contexto (conceptos de "attention decay" y "compactación" como mecanismo de resumen forzado al llenarse el contexto).

Parte 5 — Cazando una alucinación

Elegí cualquier asistente de IA real al que tengas acceso (Claude, ChatGPT, Gemini, Copilot Chat, etc.) y hacele una pregunta pensada para inducir una alucinación. Algunas ideas de consigna (podés usar una de estas o inventar la tuya, siempre que apunte a un dato verificable y poco común):

- Pedile el número exacto de un artículo de una ley poco conocida (de cualquier país).
- Pedile que te explique una función o método que no existe en una librería real que sí conocés.
- Pedile datos biográficos muy específicos (fecha, cifra, cita textual) de una persona real pero poco documentada en internet.

Documentá:

- a. El prompt exacto que usaste.
- b. La respuesta completa del modelo, tal cual la dio (captura de pantalla o copiado textual).
- c. Cómo verificaste el dato contra una fuente real, y qué parte específica de la respuesta resultó inventada (si no lograste inducir una alucinación en tres intentos, documentá los tres intentos igual y explicá por qué creés que el modelo no falló ahí).
- d. Una reflexión de 3-4 líneas: ¿la respuesta incorrecta sonaba tan segura y fluida como una correcta? ¿Cómo se conecta esto con la idea de que la confiabilidad no es una propiedad fija del modelo, sino algo que el desarrollador construye verificando la salida?

Parte 6 — Laboratorio de tokenización con un modelo real

Usá la herramienta pública y gratuita de OpenAI para contar tokens —no hace falta cuenta ni API key—: **Tokenizer**, disponible en platform.openai.com/tokenizer. Seleccioná el modo **GPT-4o & GPT-4o mini** (tokenizador `o200k_base`) para todo el laboratorio, así los tres ejercicios son comparables entre sí.

Si no podés acceder a esa herramienta, una alternativa equivalente es la librería `tiktoken` de OpenAI (open source, se puede correr localmente en Python) — avisá a la cátedra si usás esta vía en vez de la web, para que quede aclarado en tu entrega.

- a. **Texto propio en español:** tomá un párrafo de ~100-150 palabras (puede ser un fragmento de tus propias respuestas del TP1, o un párrafo de cualquier documento de esta unidad) y pegalo en el Tokenizer. Reportá:

cantidad de *tokens*, cantidad de *palabras* y cantidad de *caracteres* que muestra la herramienta, y calculá la razón tokens/palabra (por ejemplo: 143 tokens / 112 palabras $\approx$ 1,28 tokens por palabra). Adjuntá la captura de pantalla.

- b. **El mismo texto en inglés:** traducí ese mismo párrafo al inglés (podés pedirle la traducción a un LLM, aclarando que lo hiciste) y tokenízalo también. Reportá los mismos tres números y compará: ¿qué versión consumió más tokens para decir lo mismo? Conectá el resultado con lo visto en la unidad sobre por qué el español (y la mayoría de los idiomas no ingleses) suele consumir más tokens que el inglés para el mismo contenido. Adjuntá la captura.
- c. **Un fragmento de código:** tokenizá una función corta (10-15 líneas, en cualquier lenguaje que conozcas) y reportá tokens y caracteres. Calculá la razón tokens/carácter y compárala contra la del texto en prosa del punto (a) — ¿el código consume más o menos tokens por carácter que el texto natural? Adjuntá la captura.

CONSIDERACIONES

- ✓ **Un solo archivo:** entregá las seis partes en un único documento, en el mismo orden de esta consigna, con las capturas de pantalla pedidas incrustadas donde corresponda (no como archivos sueltos).
- ✓ **Evidencia real, no inventada:** los números de tokens/palabras/caracteres de la Parte 6 tienen que salir de la herramienta real — no se aceptan valores estimados a ojo sin la captura que los respalde.
- ✓ **Fuentes citables:** todo dato de documentación oficial (tabla de modelos de la Parte 2, ventana de contexto y manejo de compactación de la Parte 4) tiene que venir con su fuente (URL o documento).
- ✓ **No hay una única respuesta correcta** en las Partes 1, 2, 3 y 5: se evalúa la calidad de la justificación técnica, no coincidir con una clasificación exacta.

CONCLUSIONES ESPERADAS

- Distinguir con criterio técnico qué es (y qué NO es) un LLM, y separar la marca/proveedor del modelo puntual al elegir qué usar para cada tarea.
- Identificar a qué categoría pertenece una herramienta de asistencia al desarrollo según su nivel de autonomía, usando el criterio simple de si tiene o no tiene "tools" para actuar sobre el entorno.
- Manejar la ventana de contexto como una magnitud concreta y medible, entendiendo además que la degradación de calidad (attention decay) y la compactación son problemas distintos de simplemente "quedarse sin espacio".
- Haber verificado en carne propia que un modelo puede alucinar con total seguridad discursiva, y qué implica eso para la confiabilidad de lo que produce.
- Entender de forma tangible, con números propios, que un token no es una palabra — y que esa diferencia tiene consecuencias reales de costo y de idioma.